

Expressivity of Description Logics and the Corresponding OWL Profiles

Abstract

This document gives a letter-by-letter account of Description Logic (DL) naming conventions. For each label it provides a description of the constructor, an example axiom in DL syntax, and the corresponding OWL 2 encoding in Turtle. It closes with the mapping between named DLs and OWL species, with a comparison of the three OWL 2 profiles, with an analysis of the rule-based entailment regimes offered by GraphDB — including an empirical check of what each ruleset actually infers —, with a case study assessing the ECMO 0.3.0 ontology against those regimes, and with a survey of openly available reasoners.

DL identifiers at a glance

Id	Construct	DL syntax	OWL 2 vocabulary
<i>Base languages</i>			
$\mathcal{AL}$	attributive language	$\top, \perp, \neg A, C \sqcap D, \forall R.C, \exists R.\top$	intersectionOf, allValuesFrom, ...
$\mathcal{EL}$	existential language	$\top, C \sqcap D, \exists R.C$	intersectionOf, someValuesFrom
$\mathcal{FL}_0$	frame language	$C \sqcap D, \forall R.C$	intersectionOf, allValuesFrom
$\mathcal{S}$	$\mathcal{ALC}$ + transitive roles	$\mathcal{ALC}, \text{Trans}(R)$	TransitiveProperty
<i>Constructor letters</i>			
$\mathcal{C}$	complex negation	$\neg C$	complementOf
$\mathcal{U}$	union	$C \sqcup D$	unionOf
$\mathcal{E}$	full existential	$\exists R.C$	someValuesFrom
$\mathcal{H}$	role hierarchy	$R \sqsubseteq S$	subPropertyOf
$\mathcal{R}$	complex role inclusions	$R_1 \circ R_2 \sqsubseteq S, \exists R.\text{Self}$	propertyChainAxiom, hasSelf
$\mathcal{O}$	nominals	$\{a, b\}$	oneOf, hasValue
$\mathcal{I}$	inverse roles	R^-	inverseOf
$\mathcal{N}$	unqualified cardinality	$\geq n R, \leq n R$	min/maxCardinality
$\mathcal{Q}$	qualified cardinality	$\geq n R.C, \leq n R.C$	min/maxQualifiedCardinality
$\mathcal{F}$	functional roles	$\top \sqsubseteq \leq 1 R$	FunctionalProperty
$(\mathcal{D})$	datatypes	$\exists U.d, \text{facets}$	DatatypeProperty, withRestrictions
<i>Families</i>			
<i>DL-Lite</i>	FOL-rewritable fragments	$B \sqsubseteq C, B ::= A \mid \exists R$	OWL 2 QL (<i>DL-Lite_R</i>)

Contents

1	How the naming works	2
2	Base languages	2
3	Constructor letters	3
4	The <i>DL-Lite</i> family	6
5	Named DLs and their OWL counterparts	8
6	The three OWL 2 profiles	8
7	Rule-based entailment in triple stores: the GraphDB rulesets	9
8	Case study: ECMO 0.3.0 against the GraphDB rulesets	12
9	Openly available reasoners	14

1 How the naming works

A DL name is built by concatenating letters: a *base language* ($\mathcal{AL}$, $\mathcal{EL}$, $\mathcal{FL}$, or the shorthand $\mathcal{S}$) followed by one letter per additional constructor. Thus

$$SROIQ^{(\mathcal{D})} = \underbrace{\mathcal{S}}_{\mathcal{ALC}+\text{trans.}} + \underbrace{\mathcal{R}}_{\text{role axioms}} + \underbrace{\mathcal{O}}_{\text{nominals}} + \underbrace{\mathcal{I}}_{\text{inverses}} + \underbrace{\mathcal{Q}}_{\text{qual. card.}} + \underbrace{(\mathcal{D})}_{\text{datatypes}}.$$

Throughout, A denotes an atomic concept (class name), C, D arbitrary concept expressions, R, S roles (properties), and a, b individuals. All Turtle snippets assume the usual `rdf:`, `rdfs:`, `owl:`, `xsd:` prefixes and a local prefix `:`.

2 Base languages

$\mathcal{FL}_0$ / $\mathcal{FL}^-$ — frame-based languages

$\mathcal{FL}_0$ offers conjunction and value restriction only; $\mathcal{FL}^-$ adds the limited existential $\exists R.T$. Both are of mainly historical interest and neither corresponds to a named OWL fragment.

$$\text{Parent} \sqsubseteq \text{Person} \sqcap \forall \text{hasChild}.\text{Person}$$

$\mathcal{AL}$ — Attributive Language

The classical baseline: atomic concepts, $\top$, $\perp$, *atomic* negation (only $\neg A$ for a class name A), conjunction, value restriction $\forall R.C$, and *unqualified* existential $\exists R.T$.

$$\text{Parent} \sqsubseteq \text{Person} \sqcap \neg \text{Robot} \sqcap \forall \text{hasChild}.\text{Person} \sqcap \exists \text{hasChild}.\top$$

```
:Parent rdfs:subClassOf [  
  a owl:Class ;  
  owl:intersectionOf (  
    :Person  
    [ a owl:Class ; owl:complementOf :Robot ]  
    [ a owl:Restriction ; owl:onProperty :hasChild ;  
      owl:allValuesFrom :Person ]  
  )  
]
```

```
[ a owl:Restriction ; owl:onProperty :hasChild ;
  owl:someValuesFrom owl:Thing ]
)
```

$\mathcal{EL}$ — Existential Language

$\top$, conjunction, and *full* existential restriction $\exists R.C$. No negation, no disjunction, no value restriction. This is the base of OWL 2 EL.

$$\text{Parent} \equiv \text{Person} \sqcap \exists \text{hasChild}.\text{Person}$$

```
:Parent owl:equivalentClass [
  a owl:Class ;
  owl:intersectionOf (
    :Person
    [ a owl:Restriction ; owl:onProperty :hasChild ;
      owl:someValuesFrom :Person ]
  )
]
```

3 Constructor letters

$\mathcal{C}$ — complex (full) concept negation

Negation of arbitrary concept expressions, not just of class names. $\mathcal{AL} + \mathcal{C} = \mathcal{ALC}$, which is closed under the Boolean operators; hence $\mathcal{U}$ and $\mathcal{E}$ come for free.

$$\text{Childless} \equiv \neg \exists \text{hasChild}.\top$$

```
:Childless owl:equivalentClass [
  a owl:Class ;
  owl:complementOf [ a owl:Restriction ;
    owl:onProperty :hasChild ;
    owl:someValuesFrom owl:Thing ]
]
```

$\mathcal{U}$ — concept union

Disjunction of concept expressions. Redundant once $\mathcal{C}$ is available (De Morgan), which is why it rarely appears in modern names.

$$\text{Person} \equiv \text{Man} \sqcup \text{Woman}$$

```
:Person owl:equivalentClass [
  a owl:Class ; owl:unionOf ( :Man :Woman )
]
```

$\mathcal{E}$ — full existential restriction

$\exists R.C$ with an arbitrary filler, as opposed to $\mathcal{AL}$'s $\exists R.\top$. Also redundant given $\mathcal{C}$ and $\forall$.

$$\text{Researcher} \sqsubseteq \exists \text{worksFor}.\text{Institution}$$

```
:Researcher rdfs:subClassOf [
  a owl:Restriction ; owl:onProperty :worksFor ;
  owl:someValuesFrom :Institution
]
```

$\mathcal{S}$ — shorthand for $\mathcal{ALC}$ with transitive roles

Not a constructor but an abbreviation for $\mathcal{ALC}_{R^+}$. Nearly every practically relevant DL name starts with it.

$$\text{hasAncestor} \circ \text{hasAncestor} \sqsubseteq \text{hasAncestor}$$

```
| :hasAncestor a owl:TransitiveProperty .
```

$\mathcal{H}$ — role hierarchy

Subsumption between roles.

$$\text{hasMother} \sqsubseteq \text{hasParent}$$

```
| :hasMother rdfs:subPropertyOf :hasParent .
```

$\mathcal{R}$ — complex role inclusions (subsumes $\mathcal{H}$)

General role chains, subject to a regularity (acyclicity) condition, plus reflexive, irreflexive and disjoint roles and local reflexivity $\exists R.\text{Self}$.

$$\text{hasParent} \circ \text{hasParent} \sqsubseteq \text{hasGrandparent}$$

$$\text{Disj}(\text{hasParent}, \text{hasSpouse})$$

$$\text{Narcissist} \equiv \exists \text{loves}.\text{Self}$$

```
| :hasGrandparent owl:propertyChainAxiom ( :hasParent :hasParent ) .
| :hasParent owl:propertyDisjointWith :hasSpouse .
| :hasPart a owl:ReflexiveProperty .
| :hasParent a owl:IrreflexiveProperty .

| :Narcissist owl:equivalentClass [
  a owl:Restriction ; owl:onProperty :loves ;
  owl:hasSelf "true"^^xsd:boolean
] .
```

$\mathcal{O}$ — nominals

Individuals used inside concept expressions: enumerations $\{a, b, c\}$ and the singleton case $\exists R.\{a\}$.

$$\text{Continent} \equiv \{\text{Africa}, \text{Americas}, \text{Antarctica}, \text{Asia}, \text{Europe}, \text{Oceania}\}$$

$$\text{Italian} \equiv \exists \text{hasNationality}.\{\text{Italy}\}$$

```
| :Continent owl:equivalentClass [
  a owl:Class ;
  owl:oneOf ( :Africa :Americas :Antarctica :Asia :Europe :Oceania )
] .

| :Italian owl:equivalentClass [
  a owl:Restriction ; owl:onProperty :hasNationality ;
  owl:hasValue :Italy
] .
```

$\mathcal{I}$ — inverse roles

R^- as a role expression, usable wherever a role name is.

$$\begin{aligned}\text{hasChild} &\equiv \text{hasParent}^- \\ \text{Child} &\sqsubseteq \exists \text{hasChild}^-. \text{Person}\end{aligned}$$

```
:hasChild owl:inverseOf :hasParent .  
  
:Child rdfs:subClassOf [  
  a owl:Restriction ;  
  owl:onProperty [ owl:inverseOf :hasChild ] ;  
  owl:someValuesFrom :Person  
] .
```

$\mathcal{N}$ — unqualified cardinality restrictions

$\geq n R$, $\leq n R$, $= n R$ with no filler (implicitly $\top$).

$$\text{Person} \sqsubseteq \leq 2 \text{ hasParent}$$

```
:Person rdfs:subClassOf [  
  a owl:Restriction ; owl:onProperty :hasParent ;  
  owl:maxCardinality "2"^^xsd:nonNegativeInteger  
] .
```

$\mathcal{Q}$ — qualified cardinality restrictions

$\geq n R.C$: the filler is constrained. Strictly more expressive than $\mathcal{N}$ (which is $\mathcal{Q}$ with $C = \top$). Introduced in OWL 2.

$$\text{Hand} \sqsubseteq \geq 5 \text{ hasPart.Finger}$$

```
:Hand rdfs:subClassOf [  
  a owl:Restriction ; owl:onProperty :hasPart ;  
  owl:minQualifiedCardinality "5"^^xsd:nonNegativeInteger ;  
  owl:onClass :Finger  
] .
```

$\mathcal{F}$ — functional roles

The special case $\top \sqsubseteq \leq 1 R$. Weaker than $\mathcal{N}$; present in *SHLF* (OWL Lite) precisely to keep complexity down.

$$\begin{aligned}\top &\sqsubseteq \leq 1 \text{ hasBiologicalMother} \\ \top &\sqsubseteq \leq 1 \text{ hasSSN}^-\end{aligned}$$

```
:hasBiologicalMother a owl:FunctionalProperty .  
:hasSSN a owl:InverseFunctionalProperty .
```

($\mathcal{D}$) — concrete domains / datatypes

Datatype properties and datatype ranges, including facet restrictions.

$$\text{Adult} \equiv \text{Person} \sqcap \exists \text{hasAge}.(\geq 18)$$

```

:Adult owl:equivalentClass [
  a owl:Class ;
  owl:intersectionOf (
    :Person
    [ a owl:Restriction ; owl:onProperty :hasAge ;
      owl:someValuesFrom [
        a rdfs:Datatype ;
        owl:onDatatype xsd:integer ;
        owl:withRestrictions ( [ xsd:minInclusive 18 ] )
      ]
    ]
  )
] .

```

4 The *DL-Lite* family

DL-Lite does not fit the letter-concatenation scheme of the previous sections, and that is deliberate. The families above were obtained *top-down*, by starting from a rich logic and asking how much can be retained while decidability holds. *DL-Lite* was designed *bottom-up* (Calvanese et al., *JAR* 2007) against a single criterion: conjunctive query answering must remain *first-order rewritable*, i.e. reducible to the evaluation of an SQL query over the data alone. Every restriction below follows from that criterion rather than from a menu of constructors.

Syntax

The language is built from *basic* concepts and roles:

$B ::= A \mid \exists R$	basic concept
$R ::= P \mid P^-$	basic role
$C ::= B \mid \neg B$	general concept
$E ::= R \mid \neg R$	general role

Here $\exists R$ is *unqualified*: it abbreviates $\exists R.\top$. A TBox contains inclusions $B \sqsubseteq C$ (and, in the extensions below, $R \sqsubseteq E$); an ABox contains assertions $A(a)$ and $P(a, b)$.

The asymmetry is the essential point. Only a *basic* concept may occur on the left of $\sqsubseteq$, and negation may occur only on the right. This guarantees that the ontology has a canonical model that can be folded into the query instead of being materialised over the data.

*DL-Lite*_{core} — concept inclusions and disjointness

The common core: inclusions between basic concepts, plus disjointness. Domain and range assertions are not primitives but derived readings of $\exists P \sqsubseteq A$ and $\exists P^- \sqsubseteq A$.

$\text{Professor} \sqsubseteq \exists \text{teaches}$	every professor teaches something
$\exists \text{teaches} \sqsubseteq \text{Professor}$	domain of teaches
$\exists \text{teaches}^- \sqsubseteq \text{Course}$	range of teaches
$\text{Professor} \sqsubseteq \neg \text{Student}$	disjointness

```

:Professor rdfs:subClassOf [
  a owl:Restriction ; owl:onProperty :teaches ;
  owl:someValuesFrom owl:Thing
] .

:teaches rdfs:domain :Professor ;

```

```

    rdfs:range    :Course .
    :Professor owl:disjointWith :Student .

```

Strictly, $DL-Lite_{\mathcal{R}}$ has only the unqualified $\exists R$. OWL 2 QL nevertheless admits a qualified existential $\exists R.A$ in superclass position, because it is a conservative extension: $A \sqsubseteq \exists R.B$ can be rewritten with a fresh role as $A \sqsubseteq \exists R_B$, $R_B \sqsubseteq R$, $\exists R_B^- \sqsubseteq B$.

$DL-Lite_{\mathcal{F}}$ — functionality

Adds functionality assertions (funct R) on basic roles, inverses included. No role inclusions.

$$\begin{array}{ll} \top \sqsubseteq \leq 1 \text{ hasSupervisor} & (\text{funct hasSupervisor}) \\ \top \sqsubseteq \leq 1 \text{ hasMatricula}^- & (\text{funct hasMatricula}^-) \end{array}$$

```

:hasSupervisor a owl:FunctionalProperty .
:hasMatricula a owl:InverseFunctionalProperty .

```

$DL-Lite_{\mathcal{R}}$ — role inclusions

Adds inclusions between roles, $R \sqsubseteq E$, hence both role hierarchies and role disjointness. No functionality. This is the logic underlying OWL 2 QL.

$$\begin{array}{l} \text{hasSupervisor} \sqsubseteq \text{knows} \\ \text{hasSupervisor}^- \sqsubseteq \text{supervises} \\ \text{hasSupervisor} \sqsubseteq \neg \text{hasStudent} \end{array}$$

```

:hasSupervisor rdfs:subPropertyOf :knows ;
               owl:propertyDisjointWith :hasStudent .
:supervises owl:inverseOf :hasSupervisor .

```

$DL-Lite_{\mathcal{A}}$ — combining the two, safely

Putting $\mathcal{F}$ and $\mathcal{R}$ together without care destroys the guarantee: with unrestricted interaction between functionality and role inclusions, conjunctive query answering is no longer FOL-rewritable and reasoning becomes ExpTime-hard (Artale et al., *JAIR* 2009). $DL-Lite_{\mathcal{A}}$ restores rewritability with a syntactic proviso:

a functional role may never be specialised — if (funct R) or (funct R^-), then R and R^- must not occur on the right-hand side of any role inclusion.

$DL-Lite_{\mathcal{A}}$ also separates abstract objects from data values, distinguishing concepts and roles from *attributes* and *value-domains* — the direct ancestor of OWL’s split between object and datatype properties.

This explains a design choice that otherwise looks arbitrary: OWL 2 QL keeps role inclusions and drops `owl:FunctionalProperty` and `owl:InverseFunctionalProperty` altogether, rather than attempting to police the interaction at the profile level.

The Horn and Boolean extensions

Artale et al. generalise the family along a second axis, by relaxing what may appear on the left of $\sqsubseteq$ and which Boolean operators are available. The fragments retained for OWL 2 QL are the *core* ones; the others show precisely what each relaxation costs.

The jump from AC^0 to PTime at the Horn line is the one that matters in practice: it is exactly the point at which the TBox can no longer be compiled away into the query, and a reasoner must instead touch the data.

Fragment	Satisfiability	CQ data compl.	Left-hand side of $\sqsubseteq$
$DL-Lite_{core}$, $DL-Lite_{\mathcal{F}}$, $DL-Lite_{\mathcal{R}}$	NLogSpace-complete	AC^0	a single basic concept B
$DL-Lite_{krom}$	NLogSpace-complete	AC^0	binary clauses ($B_1 \sqsubseteq \neg B_2$ etc.)
$DL-Lite_{horn}$	PTime-complete	PTime	conjunction $B_1 \sqcap \dots \sqcap B_n$
$DL-Lite_{bool}$	NP-complete	coNP	arbitrary Boolean combination

Why FOL-rewritability is the point

Given a conjunctive query q and a TBox $\mathcal{T}$, the PERFECTREF algorithm rewrites q into a union of conjunctive queries $q_{\mathcal{T}}$ such that, for every ABox $\mathcal{A}$,

$$\text{cert}(q, \langle \mathcal{T}, \mathcal{A} \rangle) = \text{ans}(q_{\mathcal{T}}, \mathcal{A}),$$

where the right-hand side evaluates $q_{\mathcal{T}}$ over $\mathcal{A}$ read as a plain relational database, with no reasoning at all. The ontology is compiled into the query, never into the data; the data may stay in an untouched legacy RDBMS behind a set of mappings. This is the basis of OBDA systems such as Mastro and Ontop, and the reason OWL 2 QL exists as a profile.

The cost is paid elsewhere: the rewriting may grow exponentially in the size of the *query* (though not of the data), which is why practical systems invest heavily in optimising and pruning $q_{\mathcal{T}}$.

5 Named DLs and their OWL counterparts

Description Logic	OWL language	Complexity (concept sat.)
$\mathcal{EL} / \mathcal{EL}^{++}$	OWL 2 EL	PTime-complete
$DL-Lite_{\mathcal{R}}$	OWL 2 QL	NLogSpace; AC^0 data compl. for CQs
DLP / pD^* -style	OWL 2 RL	PTime-complete
$\mathcal{ALC}$	—	ExpTime-complete
$\mathcal{SHIF}^{(D)}$	OWL 1 Lite	ExpTime-complete
$\mathcal{SHOIN}^{(D)}$	OWL 1 DL	NExpTime-complete
$\mathcal{SROIQ}^{(D)}$	OWL 2 DL	N2ExpTime-complete
unrestricted, RDF semantics	OWL 2 Full	undecidable

Note that OWL Lite, despite its name, is $\mathcal{SHIF}^{(D)}$: it has the same worst-case complexity as $\mathcal{SHIF}$, and most of OWL DL can be encoded into it. This is why it was effectively abandoned in OWL 2.

6 The three OWL 2 profiles

The profiles are *pairwise incomparable* — they do not form a chain. Each is a syntactic subset of OWL 2 DL designed to make a *different* reasoning task tractable.

OWL 2 EL ($\mathcal{EL}^{++}$). Target: very large terminologies (SNOMED CT, GO, NCIt). Retains $\sqcap$, $\exists R.C$, $\top$, $\perp$, nominals, role chains, **Self**, domain and range, keys, datatypes. Drops $\forall R.C$, $\sqcup$, $\neg$, inverse roles, all cardinality restrictions, functional properties, and disjunctive reasoning. Classification is polynomial.

OWL 2 QL ($DL\text{-}Lite_{\mathcal{R}}$, see Section 4). Target: query answering over large ABoxes stored in relational databases via rewriting (OBDA, Ontop). The syntax is asymmetric: on the left of $\sqsubseteq$ one may use a class name or $\exists R.\top$; on the right, a class name, $\exists R.C$, or a negation. Drops $\sqcup$, $\exists R.C$ in subclass position, property chains, functional and transitive properties, nominals, and keys. Conjunctive queries are FOL-rewritable into SQL, giving AC^0 data complexity.

OWL 2 RL. Target: rule engines and forward chaining over RDF (partial axiomatisation as Datalog). Also asymmetric: subclass position allows $\sqcap$, $\sqcup$, $\exists R.C$ and nominals; superclass position allows $\sqcap$, $\forall R.C$, $\leq 1 R.C$, $\leq 0 R.C$ and negation. Drops $\exists R.C$ on the right (no existential blank-node generation) and $\sqcup$ on the right. Keeps inverses, transitivity, functionality, property chains and keys. PTime data complexity, and it degrades gracefully: the RL rule set can be run over any RDF graph, including OWL Full ones, at the cost of incompleteness.

Construct	EL	QL	RL
$C \sqcap D$	yes	partial	yes
$C \sqcup D$	no	no	subclass position only
$\neg C$	no	superclass position only	superclass position only
$\exists R.C$	yes	superclass position only	subclass position only
$\forall R.C$	no	no	superclass position only
Nominals $\{a\}$	yes	no	yes
Inverse roles R^-	no	yes	yes
Cardinality restrictions	no	no	$\leq 1, \leq 0$ only
Functional properties	no	no	yes
Transitive properties	yes	no	yes
Property chains	yes	no	yes
$\exists R.\text{Self}$	yes	no	no
Keys	yes	no	yes
Data complexity	PTime	AC^0	PTime

7 Rule-based entailment in triple stores: the GraphDB rulesets

Triple stores rarely embed a DL reasoner. GraphDB, like most of them, offers entailment through *rulesets*: `empty`, `rdfs`, `rdfsplus`, `owl-horst`, `owl-max`, `owl2-ql` and `owl2-rl`, each also available as an `-optimized` variant. The names suggest a correspondence with the languages of the previous sections, but the correspondence is loose, for three reasons.

- **The engine is a rule engine.** Each ruleset is a set of Horn rules (in GraphDB's `.pie` format) applied by forward chaining; the closure is fully materialised at load time.
- **The semantics is RDF-based.** Rules match triples, not axioms; classes may be instances, and no Direct-Semantics interpretation is involved.
- **There is no syntactic check.** A ruleset is applied to any graph, whether or not it lies in the corresponding profile or DL.

The consequence is that none of the rulesets implements a full description logic. Each is best understood as a *Horn fragment* of some DL: sound, but incomplete with respect to the full logic whose vocabulary it accepts.

Mapping to DL families

The reference points are $\mathcal{SHIF}^{(\mathcal{D})}$ for OWL Lite, $\mathcal{SHOIN}^{(\mathcal{D})}$ for OWL 1 DL and $\mathcal{SROIQ}^{(\mathcal{D})}$ for OWL 2 DL. GraphDB ships no ruleset for OWL 2 EL.

GraphDB ruleset	Closest DL / profile	DL axioms covered
rdfs	$\rho df \subset DL-Lite_{\mathcal{R}}$	$A \sqsubseteq B, P \sqsubseteq Q, \exists P \sqsubseteq A$ (domain), $\exists P^- \sqsubseteq A$ (range)
rdfsplus owl-horst	Horn fragment of $\mathcal{SHI}$ ($\approx$ DLP) $pD^* \approx \text{Horn-}\mathcal{SHOIF}$	$+ P \equiv Q^-, P \equiv P^-$ (symmetry), $\text{Trans}(P)$ $+ a \approx b, \text{Func}(P), \text{Func}(P^-), \exists P.C \sqsubseteq A, A \sqsubseteq \forall P.C, A \equiv \exists P.\{o\}, A \equiv B$
owl-max owl2-rl	Horn fragment of $\mathcal{SHIF}^{(\mathcal{D})}$ (OWL Lite syntax) $\text{OWL 2 RL} \subset \text{Horn-}\mathcal{SROIQ}^{(\mathcal{D})}$	$+ B \sqcup C \sqsubseteq A$, intersections, $\leq 1 P$ $+ \text{property chains, keys, disjoint properties, qualified } \leq 1$
owl2-ql	$\text{OWL 2 QL} \approx DL-Lite_{\mathcal{R}}$	$A \sqsubseteq B, A \sqsubseteq \exists P.B$, role inclusions, inverses, disjointness

Why every ruleset is a Horn fragment

Two limitations are shared by all rulesets, and together they account for most of the distance from the full logics.

- **No disjunctive consequences.** A rule can derive $A(a)$ but never $A(a) \vee B(a)$. For a union $\text{Staff} \equiv \text{Academic} \sqcup \text{TechnicalStaff}$ only the direction $\text{Academic} \sqsubseteq \text{Staff}$ is usable; the converse, which a tableau would exploit by case analysis, is lost.
- **No new individuals.** Rules never invent a witness. An axiom $A \sqsubseteq \exists P.B$ is accepted and stored, but it yields no P -successor, and nothing that would follow from one.

Ruleset by ruleset

RDFS. Not a DL at all, since it permits metamodelling. Its DL-expressible part is a small subset of both $DL-Lite_{\mathcal{R}}$ and $\mathcal{EL}$.

RDFS-Plus. Adds the role axioms of $\mathcal{SHI}$ — inverse and transitive roles on top of the role hierarchy — but no new concept constructors. It lies within Description Logic Programs, the intersection of DL and Horn logic identified by Grosz et al. (WWW 2003).

OWL-Horst. Ter Horst’s pD^* (JWS 2005) was defined as a semantic extension of RDFS with an “if”-semantics for the OWL vocabulary, not as a DL. Its constructors roughly cover Horn- $\mathcal{SHOIF}$: `owl:hasValue` contributes nominals, functional and inverse-functional properties contribute $\mathcal{F}$. It is sound but incomplete for $\mathcal{SHOIN}$. GraphDB’s implementation goes beyond textbook pD^* in handling `owl:intersectionOf` in both directions (see the empirical check below).

OWL-Max. Accepts OWL Lite, i.e. $\mathcal{SHIF}^{(\mathcal{D})}$, syntax, but derives only its Horn-expressible consequences. Relative to OWL-Horst it adds, chiefly, the subclass direction of `owl:unionOf`, rule-expressible cardinality (above all ≤ 1 , which behaves as a local functionality constraint and so produces `owl:sameAs`), and further consistency checks. The exact rule difference is version-dependent; the authoritative source is a diff of the two `.pie` files.

OWL 2 RL. The W3C profile was itself derived from DLP and pD^* . On ontologies that satisfy the RL syntactic restrictions, the RL/RDF rules are sound and complete for ground atomic consequences (Theorem PR1 of the OWL 2 Profiles specification); outside them they remain sound but incomplete. Data complexity is polynomial.

OWL 2 QL. The *DL-Lite_R* profile of Section 4, designed for query rewriting with AC^0 data complexity. GraphDB does *not* rewrite: it materialises the QL consequences instead, which forgoes the main benefit of the profile and, as the empirical check shows, produces far more implicit triples than the other rulesets. The ruleset is also more permissive than the profile.

The -optimized variants

According to the GraphDB documentation, the optimised rulesets remove inferences that matter for formal completeness but are rarely queried: the axiomatic triples and rule conclusions involving `rdfs:Resource`, and a handful of further axiomatic triples (e.g. `owl:sameAs` typed as symmetric and transitive). The documented list does *not* include domain and range rules — which makes the first finding below unexpected.

Empirical check

The claims above were tested on a small university knowledge graph that exercises one construct per axiom block, loaded into four repositories (`rdfsplus-optimized`, `owl-horst-optimized`, `owl-max-optimized`, `owl2-ql-optimized`; `owl:sameAs` enabled) and queried with inference switched on. The key trick is an individual with no asserted type, whose every class membership must therefore be inferred:

```
ex:teaches a owl:ObjectProperty ;
           rdfs:domain ex:Academic ; rdfs:range ex:Course .
ex:alice   ex:teaches ex:kg101 .      # no rdf:type for ex:alice
```

	Construct tested	RDFS-Plus	Horst	Max	QL
Q1	domain, subclass	× ^a	✓	✓	✓
Q2	subproperty, range	× ^a	✓	✓	✓
Q3	inverse, symmetric	✓	✓	✓	✓
Q3	transitive	✓	✓	✓	×
Q4	equivalent class / property	— ^b	✓	✓	✓
Q5	sameAs via (inverse-)functional prop.	×	✓	✓	×
Q6	hasValue, someValuesFrom, allValuesFrom	×	✓	✓	×
Q7	intersection, subclass side	×	✓ ^c	✓	✓ ^d
Q7	intersection, superclass side	×	✓	✓	✓
Q7	union, subclass side	×	×	✓	×
Q8	property chain	×	×	×	×
Q9	keys	×	×	×	×
Q10	implicit triples	193	337	361	1596

^a unexpected, see below. ^b not observable: the query depends on the domain inference.

^c beyond textbook *pD**. ^d beyond the OWL 2 QL profile.

Most results match the analysis. The OWL-Horst and OWL-Max columns agree everywhere except on union, which confirms both the inclusion between the two and that union is the only difference this test bed exposes. Property values stated on an alias propagate across `owl:sameAs`. OWL 2 QL has no transitivity, no equality and no `hasValue`, and does not derive the qualified existential on the subclass side. Q8 and Q9 are empty everywhere simply because no `owl2-r1` repository was part of the run; that ruleset should return the chained affiliations and the key-based equality.

Four observations deviate from expectations.

1. **rdfsplus-optimized applies neither domain nor range.** Inverse, symmetric and transitive properties work, and equivalent classes are turned into subclass links, so the ruleset is active; yet `ex:alice` receives no type and `ex:kg101` is never a course. An earlier run on an RDFS repository showed the same symptom. Nothing in the documentation of the

optimised variants accounts for it. To be confirmed by querying range and subproperty inferences separately, and by repeating the run on the non-optimised ruleset.

2. **OWL-Horst supports intersection in both directions**, which textbook pD^* does not.
3. **OWL 2 QL derives membership in an intersection** (an individual typed with both conjuncts becomes an instance of the intersection class), a subclass-side use that *DL-Lite_R* does not allow.
4. **OWL 2 QL materialises almost five times as much as OWL-Max**, partly because it types every individual as `owl:Thing`, which the other optimised rulesets do not.

Practical consequences

- **Names are not guarantees.** A ruleset may be weaker than its name suggests (RDFS-Plus above) or stronger than the profile it is named after (OWL-Horst, OWL 2 QL). Test the constructs an application relies on, rather than inferring support from the ruleset name.
- **DL-illegal input is silently accepted.** An `owl:InverseFunctionalProperty` on a datatype property is outside *SHOIN* and *SRQIQ*, yet it produces `owl:sameAs` in every ruleset that handles inverse functionality.
- **The gap to a DL reasoner is measurable.** Running the same ontology through HermiT or Konclude (or ELK for its EL part) and comparing inferred class assertions isolates exactly the consequences that need disjunctive or existential reasoning.
- **Materialisation cost varies sharply between rulesets**, and is not monotone in apparent expressivity, as the QL figure shows.

8 Case study: ECMO 0.3.0 against the GraphDB rulesets

This section applies the analysis of Section 7 to a real ontology network: ECMO 0.3.0, the JRC ontology for emergency and crisis management, built on the Sendai Framework terminology and aligned to DOLCE+DnS Ultralite (DUL) through d0. The assessment is based on the backbone modules (hub, core, core pattern, foundation, disaster, risk); the remaining domain modules and the file of inferred axioms were not inspected, so the picture is indicative rather than a complete profile.

Constructs used and ruleset support

The defined class that carries most of the semantic weight is

`Disaster` $\equiv$ `HazardousEvent` $\sqcap$ $\exists$ `hasImpact`.(`Impact` $\sqcap$ $\exists$ `hasImpactIntensity`.`{VeryHighImpactIntensity}`).

Only its right-to-left half is rule-expressible, and only OWL 2 RL guarantees it; it is what lets a materialising store classify a hazardous event as a disaster from its reported impacts.

DL expressivity

The constructs above — nominals through `hasValue`, inverse and transitive roles, role chains, qualified cardinality, union in superclass position, datatypes — place ECMO in *SRQIQ^(D)*, i.e. OWL 2 DL, and outside all three OWL 2 profiles.

Strictly, one axiom takes it out of OWL 2 DL. `foundation:assesses` is defined by a property chain and is therefore a *non-simple* role, and OWL 2 DL forbids cardinality restrictions on non-simple roles:

Construct	Example in ECMO	RDFS	RDFS+	Horst	Max	QL	RL
Subclass, subproperty, domain, range	throughout	✓	✓	✓	✓	✓	✓
Inverse properties	dozens of named pairs, e.g. <code>hasHazardDeterminant</code>		✓	✓	✓	✓	✓
Transitive properties	<code>subsumedUnder</code> , <code>specializes</code> , <code>includesEvent</code>		✓	✓	✓	×	✓
Symmetric property	<code>isComplementaryTo</code>		✓	✓	✓	✓	✓
<code>hasValue</code> , superclass side	<code>DirectlyAffectedPopulation</code> $\sqsubseteq$ <code>∃hasAffectednessType.{Direct}</code>			✓	✓	×	✓
<code>owl:sameAs</code>	the HIPS hazard individuals			✓	✓	×	✓
Defined class (intersection, nested $\exists$, <code>hasValue</code>)	<code>Disaster</code> , see below			partial	partial	×	✓ ^a
Union, subclass side	<code>Eventuality</code> $\equiv$ <code>Event</code> $\sqcup$ <code>EventType</code> $\sqcup$ <code>Situation</code>			×	✓	×	✓
Union, superclass side	<code>Assessment</code> $\sqsubseteq$ <code>Eventuality</code> $\sqcup$ <code>InformationEntity</code>	×	×	×	×	×	×
$\exists$, superclass side	<code>Risk</code> $\sqsubseteq$ <code>∃hasHazardDeterminant.Hazard</code>		no witness generated				
Qualified cardinality	<code>Assessment</code> $\sqsubseteq \geq 1$ <code>assesses.Entity</code>	×	×	×	×	×	×
Property chains	<code>isSettingFor</code> $\circ$ <code>isAbout</code> $\sqsubseteq$ <code>assesses</code>					×	✓
Disjointness	<code>Frequent/InfrequentDisaster</code> ; <code>DUL: Description</code> $\sqcap$ <code>Situation</code> $\sqsubseteq \perp$		checks only (Max, RL)				
Punning	hazard, risk and disaster subtypes		transparent to rules				

^a in the classifying direction ($\sqsubseteq$) only.

```
foundation:Assessment rdfs:subClassOf [ a owl:Restriction ;
    owl:minQualifiedCardinality "1"^^xsd:nonNegativeInteger ;
    owl:onClass dul:Entity ; owl:onProperty foundation:assesses ] .
```

Since $\geq 1 R.C \equiv \exists R.C$ and existentials are permitted on non-simple roles, replacing the restriction with `owl:someValuesFrom dul:Entity` preserves the meaning and restores OWL 2 DL conformance.

What each ruleset delivers for ECMO

- **RDFS-Plus is the minimum useful regime.** ECMO’s own design rests on the Description-and-Situation subsumption hierarchy expressed through the transitive `subsumedUnder`/`specializes`, on systematically named inverses, and on domain/range typing. In view of the anomaly reported in Section 7, domain and range inference must be verified on whichever ruleset is deployed.
- **OWL-Horst** adds the `hasValue` classifications of the affected-population and risk-profile subclasses, and merges the HIPS hazard individuals through `owl:sameAs`.
- **OWL-Max** adds the union, making every event and situation a `d0:Eventuality` — largely redundant with the domain axioms on that class.
- **OWL 2 RL** is the only ruleset that fires the property chains linking `assesses` to DUL’s setting and aboutness relations, that reliably classifies events as `Disaster`, and that checks the disjointness axioms ECMO relies on as coherence guards.
- **OWL 2 QL** is unsuitable: without transitivity the whole `subsumedUnder` hierarchy is lost, together with chains and `hasValue`.
- **No ruleset** exploits the union in superclass position, the superclass-side existentials (the four determinants of `Risk`, `Signal`, `EventValidation`), or the cardinality axiom. Their consequences require a DL reasoner.

Deployment notes

- **Imports are not followed.** GraphDB does not resolve `owl:imports`; all modules, and DUL/d0, must be loaded explicitly. Without DUL the property chains have nothing to fire on and the DUL disjointness guards are absent.
- **The import closure is incomplete as distributed.** The catalog maps the eight alignment modules to a `0.3.0-alignments/` directory that is not present, and the `frames` module imported by `ecmo-core` has no catalog entry (presumably `ecmo-frame-model.ttl`).
- **Recommended set-up:** OWL 2 RL in the store, complemented by an offline DL reasoner (Konclude or HermiT) for the non-Horn consequences. The distributed `ecmo-inferredaxioms.ttl` appears to be the output of such a run; if so, loading it alongside the modules closes most of the remaining gap (not verified).
- **A modelling point to review.** `risk:Risk` is asserted as a subclass of `hazard:Hazard`, so every rule-based regime types every risk as a hazard — at odds with ECMO’s own reading of risk as a function of hazard, exposure, vulnerability and capacity.

A complete, rather than sampled, profile can be obtained with ROBOT once the catalog is fixed; the RL report lists exactly the axioms a GraphDB OWL 2 RL repository cannot fully exploit, the DL report should flag the cardinality violation above.

```
robot merge --catalog catalog-v001.xml --input ecmo-core.ttl --output merged.owl
robot validate-profile --profile RL --input merged.owl --output rl-violations.txt
robot validate-profile --profile DL --input merged.owl --output dl-violations.txt
```

9 Openly available reasoners

The sections above describe languages; this one describes what is actually implemented. Three warnings before the tables.

First, “supports $\mathcal{X}$ ” is a claim about the calculus, not about the code. Most systems implement a strict subset of the logic they advertise, and the gap is usually in datatypes, nominals or the more exotic role axioms. The *Notes* column flags the cases where the gap is large enough to change a tool choice.

Second, expressivity is rarely the binding constraint in practice. Maintenance is. A systematic 2023 review of the field found that fewer than half of the analysed reasoners had seen any development activity in the preceding three years, and that several of the best-known names — HermiT among them — were effectively abandoned while remaining in daily use for want of alternatives (Abicht, arXiv:2309.06888).

Third, the comparative evidence is dated. The last edition of the OWL Reasoner Evaluation competition was ORE 2015 (Parsia et al., *JAR* 2017); there has been no comparable successor, so performance claims newer than that rest on individual papers rather than on a common benchmark.

OWL 2 DL: the *SROIQ* tier

For new work at this tier the realistic choices are Konclude, which is the fastest and the only one under active development, and HermiT, which is native to the OWL API and ubiquitous but unmaintained. Everything else is either a fork of one of these or of historical interest only.

Reasoner	Expressivity	Method	Notes
Konclude	$\mathcal{SROIQV}^{(D)}$	tableau + saturation	C++/Qt, LGPLv3. $\mathcal{V}$ = nominal schemas, i.e. DL-safe rules in ontology syntax. Won most DL tracks of ORE 2015
HermiT	$\mathcal{SROIQ}^{(D)}$	hypertableau	Java, LGPL. Upstream dead since 1.3.8; the living artefact is the OWL API fork, explicitly unsupported by its authors. Ships with Protégé
FaCT++	$\mathcal{SROIQ}^{(D)}$	optimised tableau	C++, LGPL. Last commit 2017; still ships with Protégé
JFact	$\mathcal{SROIQ}^{(D)}$	tableau	Java port of FaCT++, maintained under the OWL API umbrella
Openllet	$\mathcal{SROIQ}^{(D)}$	tableau	Java. Community fork of Pellet; sporadic but real updates
Pellet	$\mathcal{SROIQ}^{(D)}$	tableau	Java. Historically important, abandoned; use Openllet instead
Sequoia	$\mathcal{SRIQ}$	consequence-based	Scala, GPL-3. No nominals — the trade for consequence-based scalability at high expressivity
Racer	$\mathcal{SRIQ}^{(D)}$	tableau	Common Lisp, BSD-3. Source on GitHub, project abandoned
Vampire	FOL	superposition	C++. A general theorem prover; OWL DL reached by translation, not a native OWL tool

OWL 2 EL: the $\mathcal{EL}^{++}$ tier

ELK’s own documentation is unusually honest about its boundaries: the supported fragment is $\mathcal{EL}^{++}$ restricted to *safe* nominals, with only partial support for concrete roles, `ObjectComplementOf` and `ObjectUnionOf`, and it warns about unsupported constructs at load time rather than failing silently. An ontology that is valid OWL 2 EL on paper may therefore still be classified incompletely — worth checking before attributing a missing inference to a modelling error.

OWL 2 QL: query rewriting and OBDA

This tier is effectively a single-vendor landscape: Ontop is the only openly available, maintained implementation of the rewriting approach described in Section 4.

OWL 2 RL: rules and materialisation

Hybrid and pay-as-you-go systems

A fourth group does not fit the profile tiers: these systems combine reasoners rather than implementing a calculus.

- **PAGOdA** answers conjunctive queries over OWL 2 DL by handling the RL-shaped bulk with a Datalog engine (RDFox) and delegating only the residual hard part to a fully-fledged DL reasoner (HermiT). Complete where the data allows, and fast where it matters.
- **MORe** and **Chainsaw** split an ontology into modules and route each to the cheapest adequate reasoner — typically ELK for the EL part and HermiT for the remainder. Both are now unmaintained.

Reasoner	Expressivity	Method	Notes
ELK	$\mathcal{EL}^{++}$, safe nominals	concurrent consequence-based	Java, Apache 2.0. Partial support only for datatypes, $\neg$ and $\sqcup$; incremental reclassification. The de-facto standard for biomedical terminologies
jcel	$\mathcal{EL}^+$	completion rules	Java. Subset of OWL 2 EL, Protégé plugin
CEL	$\mathcal{EL}^+$	completion rules	The original large-scale EL reasoner; now of mainly historical interest
ElepHant	$\mathcal{EL}^+$	consequence-based	C. Built for small memory and CPU budgets, e.g. embedded use
Whelk	$\mathcal{EL} + \text{RL}$	consequence-based	Scala. Designed for large EL TBoxes combined with RL-shaped ABoxes; integrated with ROBOT in the OBO toolchain
Snorocket	OWL 2 EL subset	completion rules	Java. Adds concrete domains; developed around SNOMED CT tooling

Reasoner	Expressivity	Method	Notes
Ontop	$DL\text{-}Lite_{\mathcal{R}} + \text{RDFS}$	rewriting to SQL	Java, Apache 2.0, actively developed (5.5.0, February 2026). SPARQL over virtual knowledge graphs via R2RML mappings; the reference OBDA system
Mastro	$DL\text{-}Lite_{\mathcal{A}}$	rewriting to SQL	Sapienza. Academic licence, not open source
Clipper	Horn- $SHIQ$	rewriting to Datalog	Java. Above QL in expressivity but still rewritable; abandoned, and its Datalog back end (DLV) is no longer distributed
Requiem	$\mathcal{ELHIO}$	resolution-based rewriting	Prototype accompanying the rewriting algorithm; not maintained

- **TrOWL** approximates an OWL 2 DL ontology into QL or EL, trading completeness for tractability in a principled way.

Choosing one

Expressivity narrows the field; the remaining criteria usually decide it.

- **Large terminology, EL-shaped** — ELK. Nothing else is competitive on SNOMED-scale classification, and the incremental mode changes the editing workflow.
- **Full OWL 2 DL, performance matters** — Konclude. If the deployment must be pure Java and OWL API-native, HermiT, with the maintenance caveat understood.
- **Data in an RDBMS, ontology used as a query interface** — Ontop.
- **Large RDF graphs, forward chaining acceptable** — RDFox if the licence is acceptable, otherwise a rule engine (VLog, Arachne) or the materialisation built into the triple store.

Reasoner	Expressivity	Method	Notes
RDFox	OWL 2 RL, Datalog, SWRL	parallel materialisation	C++. Highest performance in this class, but commercial and closed source; free academic licences available
VLog	Datalog + existential rules	chase	C++/Java, open source. Rule engine rather than an OWL tool; OWL RL reached by translating axioms to rules
OWL-RL	OWL 2 RL	forward chaining	Python, on top of RDFLib. Convenient rather than fast
reasonable	OWL 2 RL subset	forward chaining	Rust, with Python bindings. Substantially faster than OWL-RL
Arachne	OWL RL	Rete	Scala. Built for Gene Ontology causal activity models
Apache Jena	RDFS, partial OWL	forward/backward rules	Java. Deliberately incomplete rule sets; ubiquitous, and often what people mean when they say “the reasoner in my triple store”
GraphDB	RDFS to OWL 2 RL / QL	forward chaining	Java. Commercial, with a free edition. Rulesets are Horn fragments and deviate from their names in both directions; see Section 7
EYE	N3 rules	Euler-path chaining	Prolog/WebAssembly, very actively developed. Not an OWL reasoner as such, but widely used for OWL-adjacent rule reasoning

- **Embedded or constrained hardware** — ElepHant (EL) or LiRoT (RL subset); both were built explicitly for that envelope.